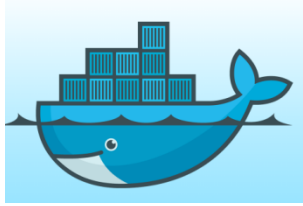


docker

Dr Ganesh Neelakanta Iyer

PROGRESS

Cargo Transport Pre-1960



Multiplicity of Goods







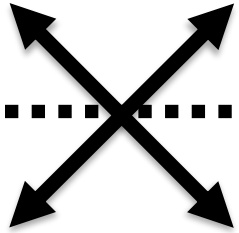








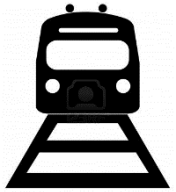
Do I worry about how goods interact (e.g. coffee beans next to spices)



Multipilicity of methods for transporting/storing

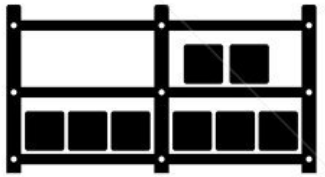








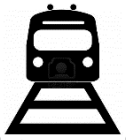
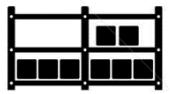




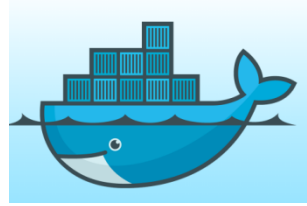


Can I transport quickly and smoothly (e.g. from boat to train to truck)

Also an NxM Matrix

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

Solution: Intermodal Shipping Container



Multiplicity of Goods



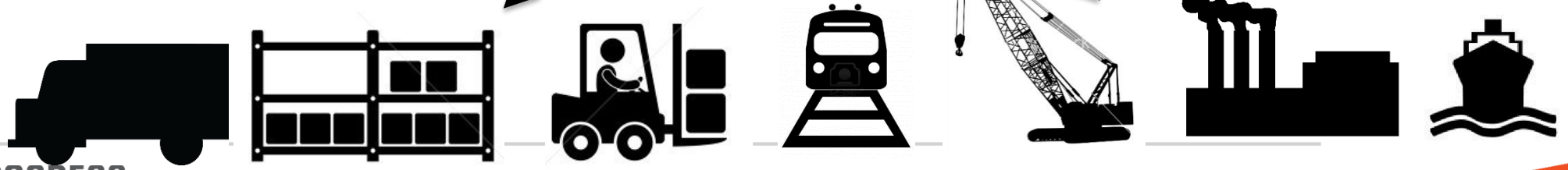
Do I worry about how goods interact (e.g. coffee beans next to spices)

A standard container that is loaded with virtually any goods, and stays sealed until it reaches final delivery.



…in between, can be loaded and unloaded, stacked, transported efficiently over long distances, and transferred from one mode of transport to another

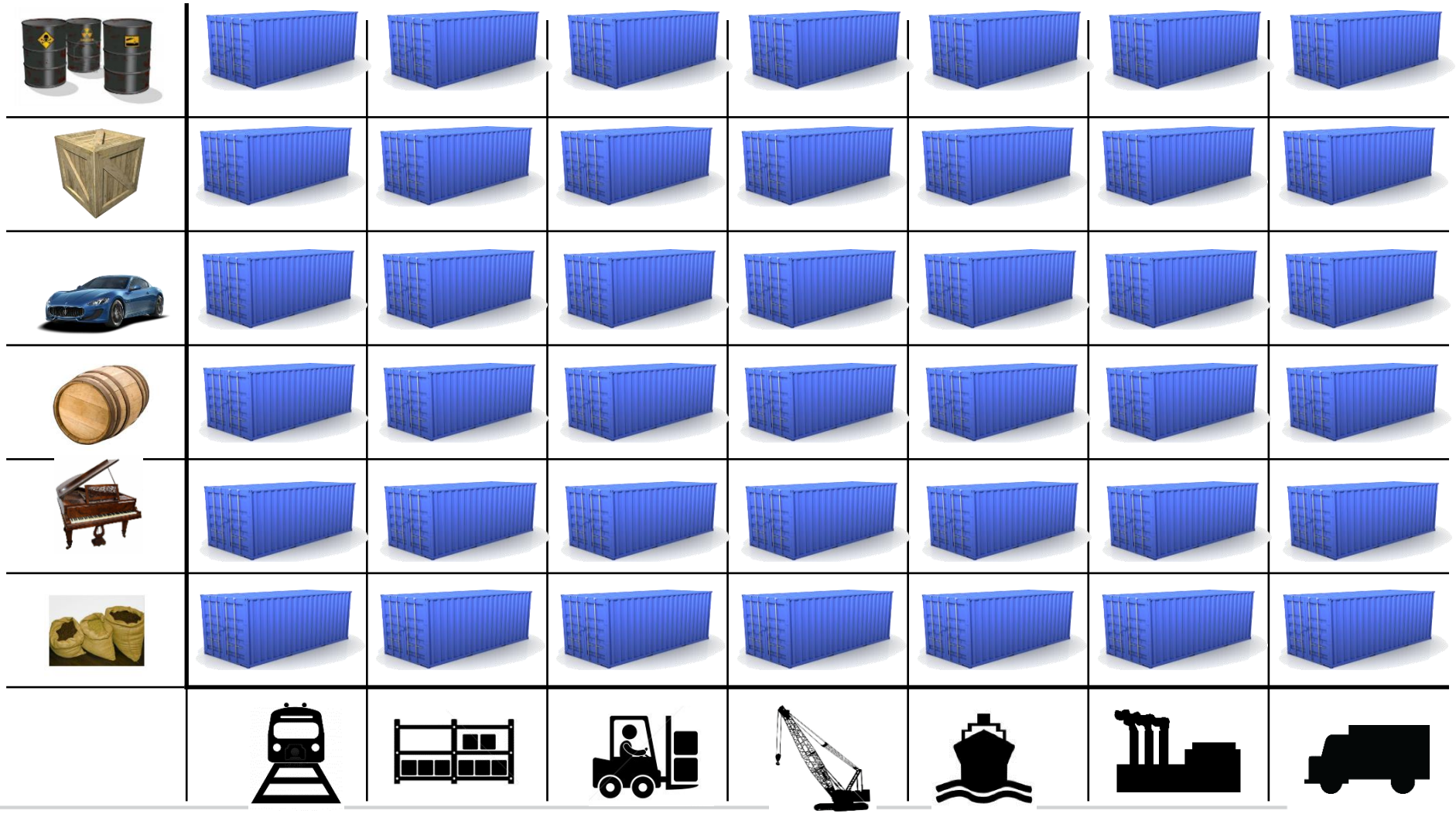
Multiplicity of methods for transporting/storing



Can I transport quickly and smoothly (e.g. from boat to train to truck)

PROGRESS

This eliminated the NXM matrix problem…



and spawned an Intermodal Shipping Container Ecosystem





- 90% of all cargo now shipped in a standard container
- Order of magnitude reduction in cost and time to load and unload ships
- Massive reduction in losses due to theft or damage
- Huge reduction in freight cost as percent of final goods (from >25% to <3%) massive globalizations
- 5000 ships deliver 200M containers per year

What is Docker?

Docker is an open-source project that automates the deployment of applications inside software containers, by providing an additional layer of abstraction and automation of operating system–level virtualization on Linux.

[Source: en.wikipedia.org]

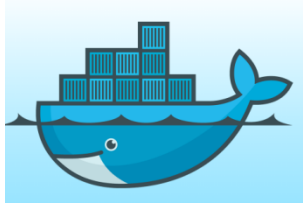
Docker allows you to package an application with all of its dependencies into a standardized unit for software development.

[www.docker.com]



http://altinvesthq.com/news/wp-content/uploads/2015/06/container-ship.jpg

The Challenge



Multiplicity of Stacks

Static website

nginx 1.5 + modsecurity + openssl + bootstrap 2

User DB

postgresql + pgv8 + v8

Queue

Redis + redis-sentinel

Analytics DB

hadoop + hive + thrift + OpenJDK

Background workers

Python 3.0 + celery + pyredis + libcurl + ffmpeg + libopencv + nodejs + phantomjs

Web frontend

Ruby + Rails + sass + Unicorn

API endpoint

Python 2.7 + Flask + pyredis + celery + psycopg + postgresql-client

Do services and apps interact appropriately?

Multiplicity of hardware environments



Development VM

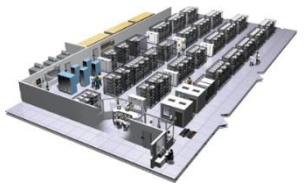


QA server

Customer Data Center



Public Cloud



Production Cluster



Disaster recovery

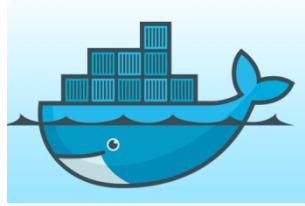
Production Servers

Contributor's laptop



Can I migrate smoothly and quickly?

Results in NXM compatibility nightmare



	Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
Static website	?	?	?	?	?	?	?
Web frontend	?	?	?	?	?	?	?
Background workers	?	?	?	?	?	?	?
User DB	?	?	?	?	?	?	?
Analytics DB	?	?	?	?	?	?	?
Queue	?	?	?	?	?	?	?



Docker is a shipping container system for code

Multiplicity of Stacks

Static website

User DB

Web frontend

Queue

Analytics DB

Do services and apps interact appropriately?

An engine that enables any payload to be encapsulated as a lightweight, portable, self-sufficient container…



…that can be manipulated using standard operations and run consistently on virtually any hardware platform

Multiplicity of hardware environments



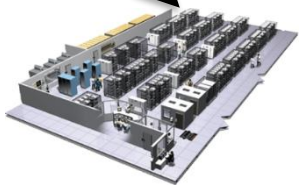
Development VM



QA server



Customer Data Center



Public Cloud



Production Cluster



Contributor's laptop

Can I migrate smoothly and quickly

Or…put more simply



Multiplicity of Stacks

Static website

User DB

Web frontend

Queue

Analytics DB

Do services and apps interact appropriately?

Developer: Build Once, Run Anywhere (Finally)



Operator: Configure Once, Run Anything

Multiplicity of hardware environments



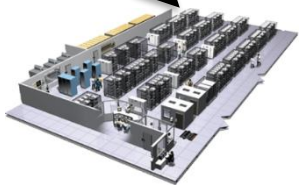
Development VM



QA server



Customer Data Center



Public Cloud



Production Cluster



Contributor's laptop

Can I migrate smoothly and quickly

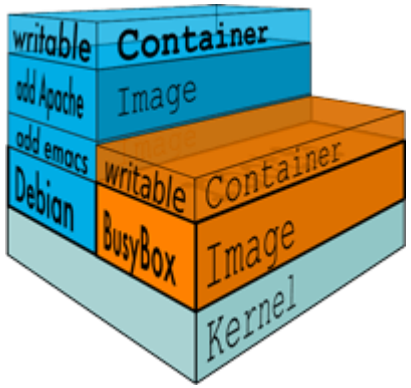
Docker solves the NXM matrix problem



PROGRESS

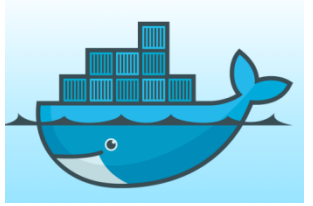
Docker containers

- Docker container wrap a piece of software in a complete file system that contains everything needed to run:
  - Code, runtime, system tools, system libraries
  - Anything that can be installed on a server



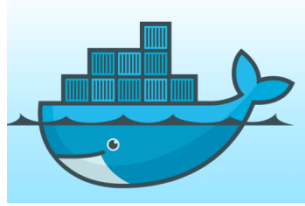
- This guarantees that the software will always run the same, regardless of the environment it is running in.

Why containers matter



	Physical Containers	Docker
Content Agnostic	The same container can hold almost any type of cargo	Can encapsulate any payload and its dependencies
Hardware Agnostic	Standard shape and interface allow same container to move from ship to train to semi-truck to warehouse to crane without being modified or opened	Using operating system primitives (e.g. LXC) can run consistently on virtually any hardware—VMs, bare metal, openstack, public IAAS, etc.—without modification
Content Isolation and Interaction	No worry about anvils crushing bananas. Containers can be stacked and shipped together	Resource, network, and content isolation. Avoids dependency hell
Automation	Standard interfaces make it easy to automate loading, unloading, moving, etc.	Standard operations to run, start, stop, commit, search, etc. Perfect for devops: CI, CD, autoscaling, hybrid clouds
Highly efficient	No opening or modification, quick to move between waypoints	Lightweight, virtually no perf or start-up penalty, quick to move and manipulate
Separation of duties	Shipper worries about inside of box, carrier worries about outside of box	Developer worries about code. Ops worries about infrastructure.

Docker containers



Lightweight

- Containers running on one machine all share the same OS kernel
- They start instantly and make more efficient use of RAM
- Images are constructed from layered file systems
- They can share common files, making disk usage and image downloads much more efficient

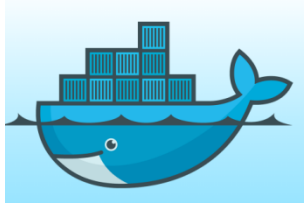
Open

- Based on open standards
- Allowing containers to run on all major Linux distributions and Microsoft OS with support for every infrastructure

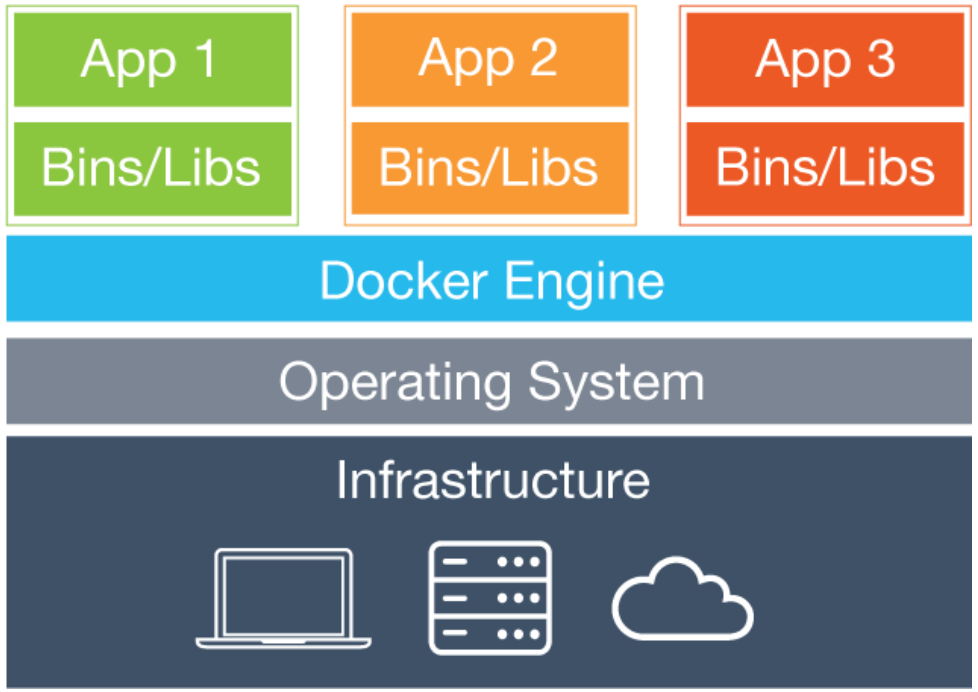
Secure

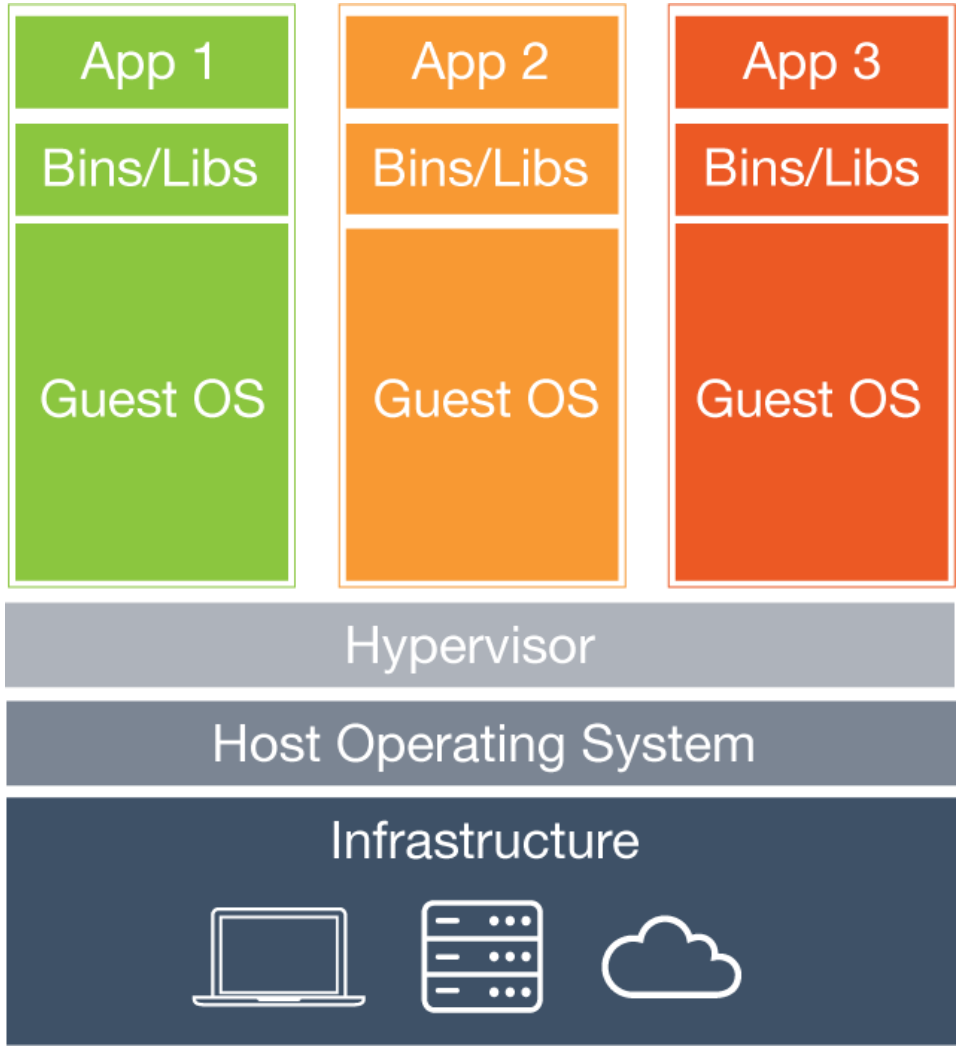
- Containers isolate applications from each other and the underlying infrastructure while providing an added layer of protection for the application

Docker / Containers vs. Virtual Machine

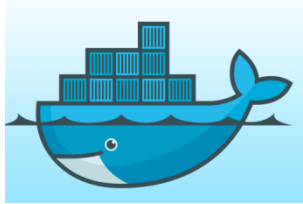


Containers have similar resource isolation and allocation benefits as VMs but a different architectural approach allows them to be much more portable and efficient

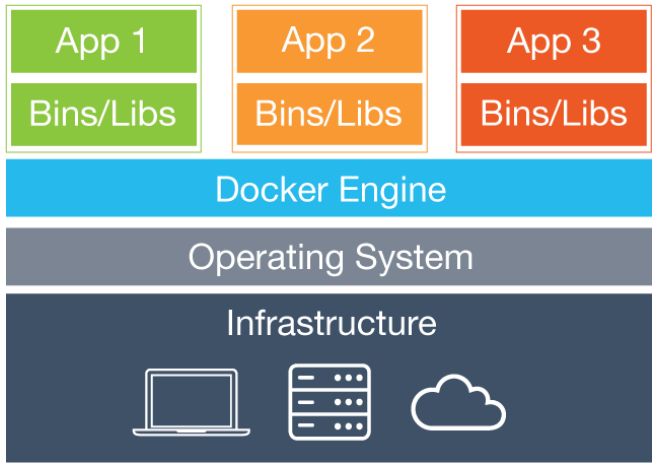




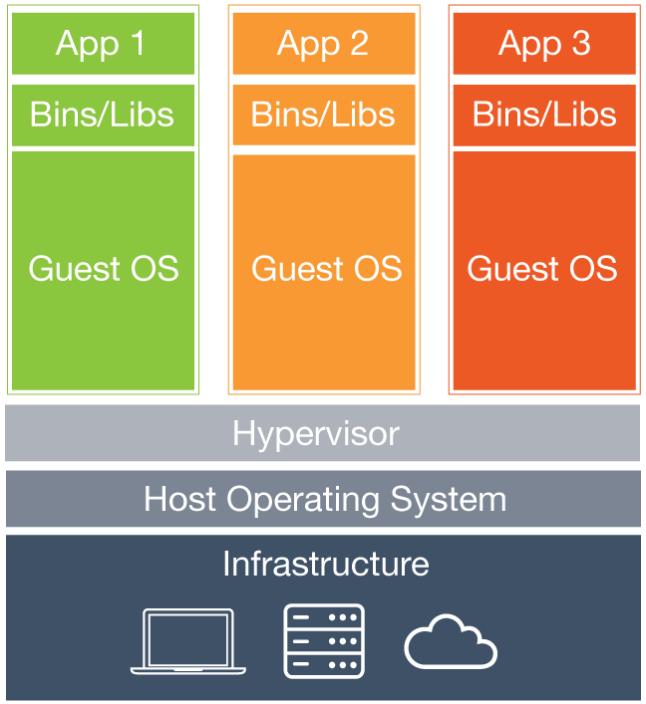
Docker / Containers vs. Virtual Machine



- It includes the application and all of its dependencies, but share the kernel with other containers.
- They run as an isolated process in userspace on the host operating system.
- Docker containers run on any computer, on any infrastructure and in any cloud



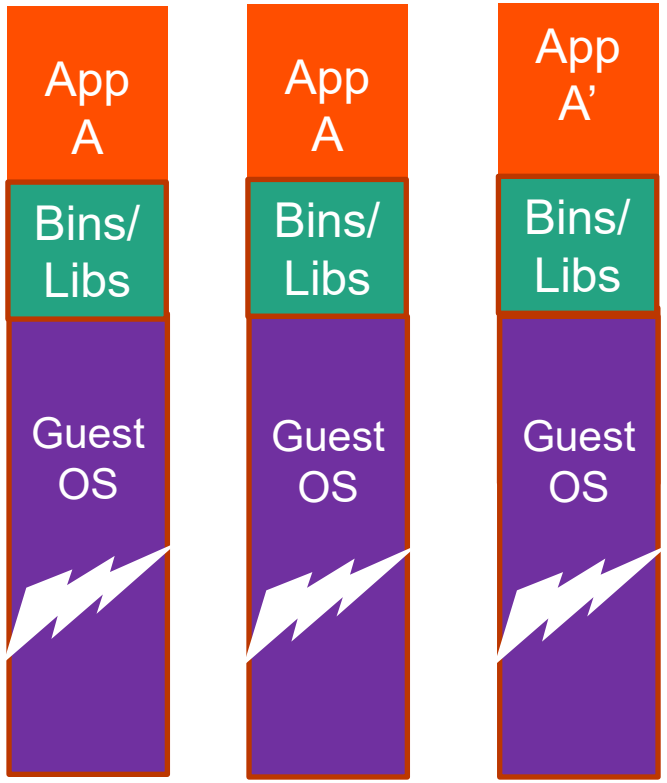
- Each virtual machine includes the application, the necessary binaries and libraries and an entire guest operating system - all of which may be tens of GBs in size



https://www.docker.com/resources/what-container

Why are Docker containers lightweight?

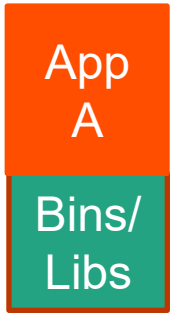
VMs



VMs

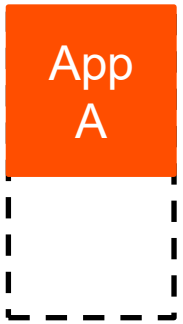
Every app, every copy of an app, and every slight modification of the app requires a new virtual server

Containers



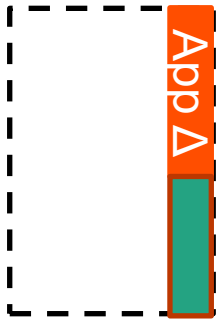
Original App

(No OS to take up space, resources, or require restart)



Copy of App

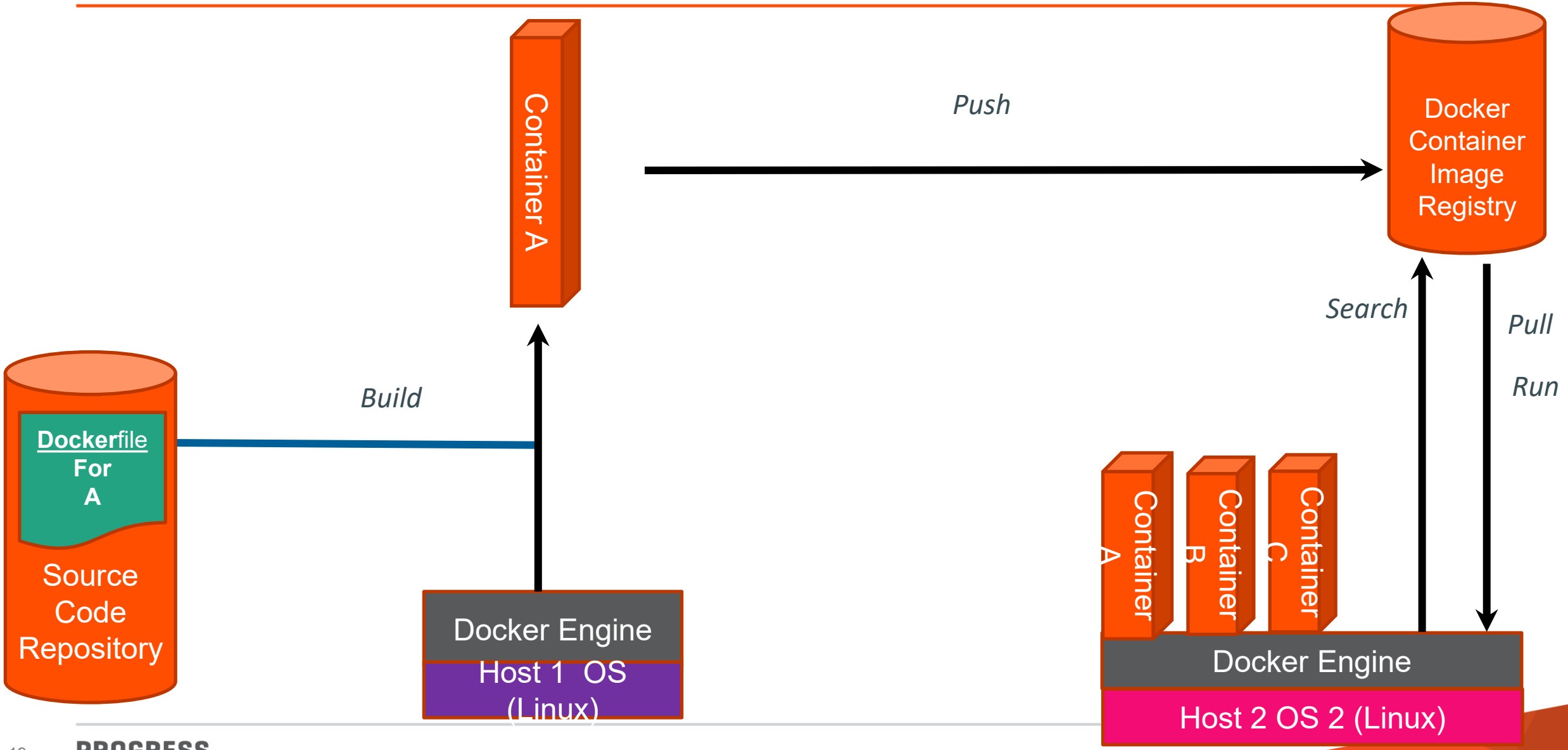
No OS. Can Share bins/libs



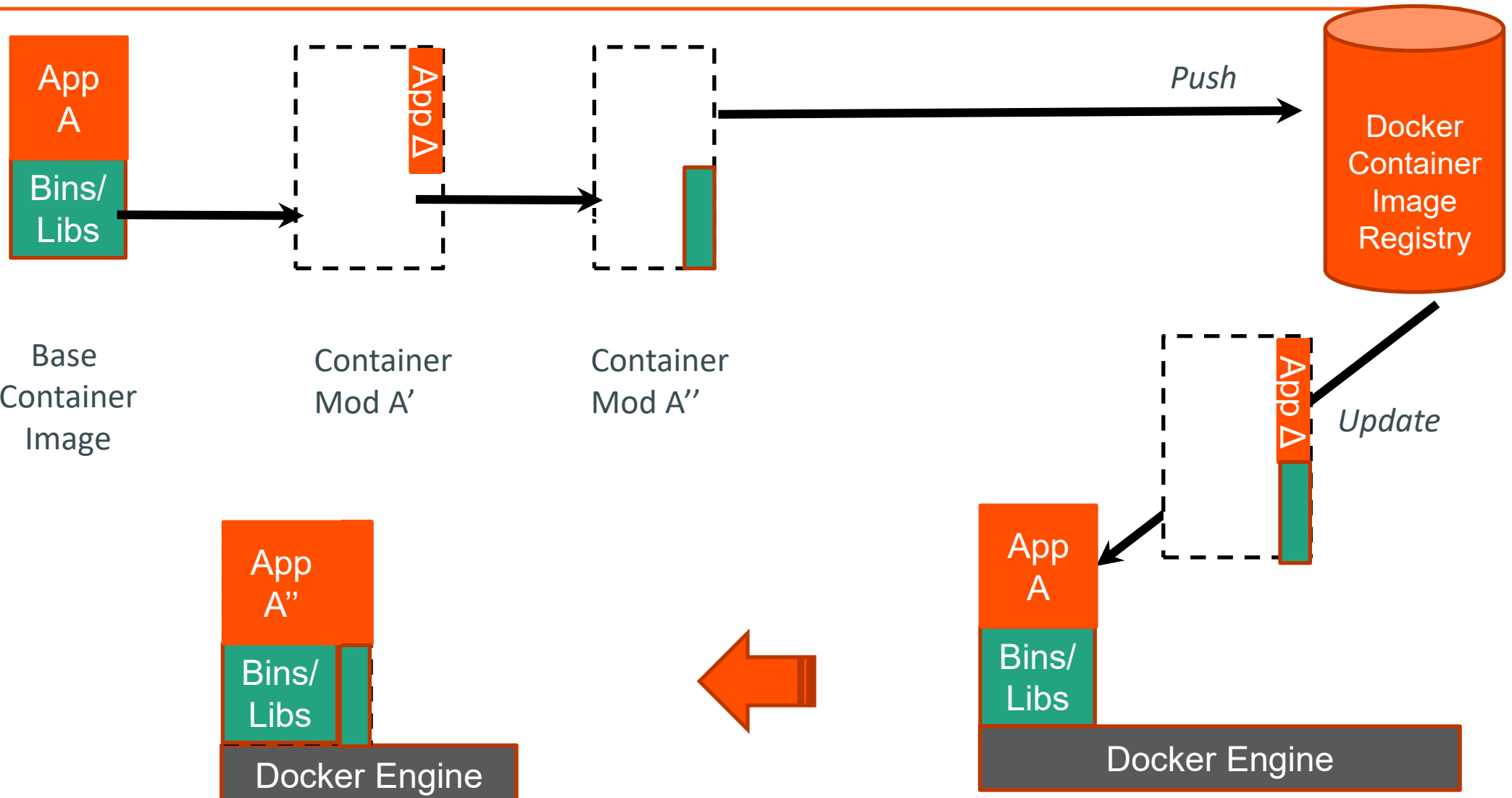
Modified App

Union file system allows us to only save the diffs Between container A and container A'

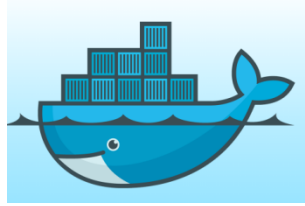
What are the basics of the Docker system?



Changes and Updates



How does this help you build better software?



Accelerate Developer Onboarding

- Stop wasting hours trying to setup developer environments
- Spin up new instances and make copies of production code to run locally
- With Docker, you can easily take copies of your live environment and run on any new endpoint running Docker.

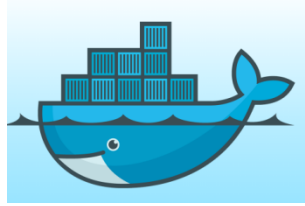
Empower Developer Creativity

- The isolation capabilities of Docker containers free developers from the worries of using “approved” language stacks and tooling
- Developers can use the best language and tools for their application service without worrying about causing conflict issues

Eliminate Environment Inconsistencies

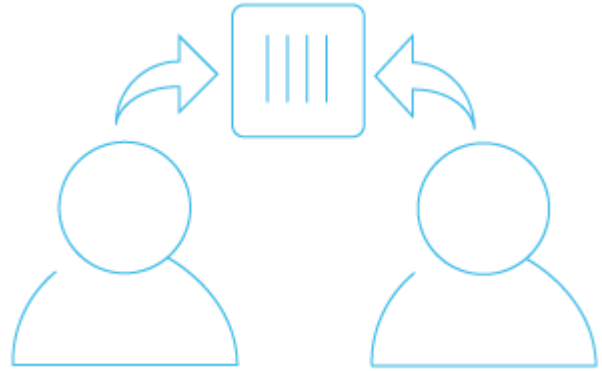
- By packaging up the application with its configs and dependencies together and shipping as a container, the application will always work as designed locally, on another machine, in test or production
- No more worries about having to install the same configs into a different environment

Easily Share and Collaborate on Applications

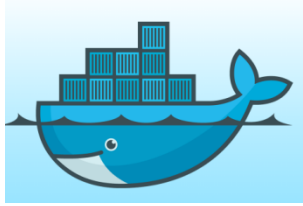


Docker creates a common framework for developers and sysadmins to work together on distributed applications

- Distribute and share content
  - Store, distribute and manage your Docker images in your Docker Hub with your team
  - Image updates, changes and history are automatically shared across your organization.
- Simply share your application with others
  - Ship your containers to others without worrying about different environment dependencies creating issues with your application.
  - Other teams can easily link to or test against your app without having to learn or worry about how it works.

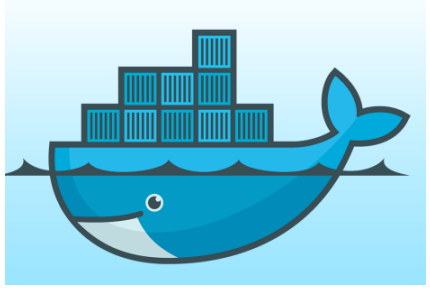


Get Started with Docker



- install Docker
- run a software image in a container
- browse for an image on Docker Hub
- create your own image and run it in a container
- create a Docker Hub account and an image repository
- create an image of your own
- push your image to Docker Hub for others to use

https://docs.docker.com/get-started



Health and fitness Mobile App

PROGRESS

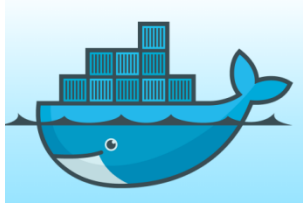
A familiar story



- You've been tasked with creating the REST API for a mobile app for tracking health and fitness
- You code the first endpoint using the development environment on your laptop
- After running all the unit tests and seeing that they passed, you check your code into the Git repository and let the QA engineer know that the build is ready for testing
- The QA engineer dutifully deploys the most recent build to the test environment, and within the first few minutes of testing discovers that your recently developed REST endpoint is broken
- After spending a few hours troubleshooting alongside the QA engineer, you discover that the test environment is using an outdated version of a third-party library, and this is what is causing your REST endpoints to break

Slight differences between development, test, stage, and production environments can wreak mayhem on an application

Solution



- Traditional approaches for dealing with this problem, such as change management processes, are too cumbersome for today's rapid build and deploy cycles
- What is needed instead is a way to transfer an environment seamlessly from development to test, eliminating the need for manual and error prone resource provisioning and configuration.
- You can create Docker images from a running container. For example, one could launch a container, install a bunch of software packages using a package manager like APT or yum, and then commit those changes to a new Docker image.

Solution → Containers (Docker)

Back to story

- First, define a Docker image for launching a container for running the REST endpoint
- Use this to test our code on a laptop, and the QA engineer can use this to test the code in Google Compute Engine/AWS EC2/Azure VM.
- The REST endpoints are going to be developed using Ruby and the Sinatra framework, so these will need to be installed in the image
- The back end will use Google Firestore/Amazon DynamoDB/Azure CosmosDB

